Culturally grounded text to image generation and evaluation for Indian language

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

Name of the Nishant (24U024011, Semester Vth)
Name of the Anuj Ram Srivastava (24U024012, Semester Vth)

Under the supervision of

Dr. Ringki Das
Assistant Professor (CSE)
Department of Computer Science and Engineering

October 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled **"Culturally grounded text to image generation and evaluation for Indian language"** has been satisfactorily completed by **Nishant (24U024011) and Anuj Ram Shrivastava (24U024012).** It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Engineering during the academic year 2026-2027.

Date:

Name of Supervisor: Dr. Ringki Das
Assistant Professor (CSE),
Computer Science and Engineering,
IIIT Bhopal (M.P.)

Department Coordinator: Dr. Rahul Chaurasia
Department of CSE
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the following major project synopsis entitled "**Culturally grounded text to image generation and evaluation for Indian language**" presented in the is the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in** Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of **Dr. Ringki Das**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Nishant (24U024011)
Sign of student

Anuj Ram Shrivastava (24U024012)
Sign of student

ACKNOWLEDGEMENT

We, Nishant and Anuj Ram Shrivastava, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled "**Culturally grounded text to image generation and evaluation for Indian language**".

First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director**, Dr. Ashutosh Kumar Singh, for providing us with the necessary resources and facilities for conducting our project. His encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, Dr. Ringki Das, for her constant guidance, invaluable suggestions, and expert advice throughout the project. Her knowledge and insights helped us enhance the quality of our work, and his/her support kept us motivated during difficult times.

Our sincere thanks to the **Department Coordinator**, Dr. Rahul Chaurasia, for the valuable support and encouragement provided during completion of the project.

We would also like to express our gratitude to all the **faculties** of the **Computer Science and Engineering** for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our **friends** for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Nishant (24U024011)
Anuj Ram Shrivastava (24U024012)

AREA OF WORK

Our research primarily involves the development of **Machine Learning, Natural Language Processing, Retrieval-Augmented Generation (RAG), and Text-to-Image Generation** using a certain suite of tools and technologies needed for building a culturally aware image generation pipeline.

Though one may claim that current text-to-image models can create an image based on textual descriptions, here comes the real problem. While the models might support multiple languages, they do not necessarily retain the cultural aspect of Indian languages such as traditional attire, festivals, rituals, decorations, and other symbols typical of regional cultures.

It has taken humans a great deal of effort since the advent of machine learning models able to process language and generate visuals. Over time, the quality of text-to-image generation improved greatly; the technology now allows creating highly realistic images. Yet the increasing diversity of languages and culture raises a new difficulty – making an image not only visually appealing but culturally accurate. It is where we aim to take things further with the use of knowledge retrieval and prompt generation.

In the course of our project, we are going to examine the architecture of a **culturally enriched prompt generation pipeline** that would be able to accept Indian-language queries and retrieve the related cultural information from certain sources such as Wikipedia and web search. Then, the acquired context would help us modify the input prompt. Furthermore, the pipeline would include the image generation using a pretrained Stable Diffusion XL (SDXL) model.

TABLE OF CONTENTS

S. No.	Title	Page No.
	Certificate	—
	Declaration	—
	Abstract	1
1	**INTRODUCTION**	**2 - 3**
1.1	Background of Text-to-Image Generation	2
1.2	Multilingual Text-to-Image Generation in Context to Indian Culture	2
1.3	Cultural Groundedness in AI-Generated Images	2
1.4	Problem Statement	3
1.5	Motivation	3
1.6	Project Objectives	3
1.7	Project Scope	3
2	**LITERATURE REVIEW**	**4-8**
2.1	Evolution of Text-to-Image Generation	4
2.2	Traditional Generative Models	4
2.3	Existing Text-to-Image Models	4-5
2.4	Indian Language Processing and Multilingual Embeddings	5
2.5	Retrieval-Augmented Generation (RAG)	6
2.6	Structured Prompt Generation	6
2.7	Existing Cultural Text-to-Image Datasets and Benchmarks	7
2.8	Comparative Analysis of Existing Approaches	7
2.9	Research Gaps Identified	8
3	**PROBLEM DEFINITION AND OBJECTIVES**	**9 - 10**
3.1	Core Problem	9
3.2	Formal Problem Statement	9

3.3	Detailed Objectives	9 - 10
3.4	Project Scope and Boundaries	10
4	**PROPOSED METHODOLOGY AND WORK DESCRIPTION**	**11 - 15**
4.1	Introduction to the Proposed Methodology	11
4.2	Overall System Architecture	11-12
4.3	Input Query Processing and Multilingual Translation	12-13
4.4	Text Representation and Cultural Knowledge Retrieval	13
4.5	Cultural Context Processing	14
4.6	Structured Prompt Generation	14-15
4.7	Experimental Methodology and Summary	15
5	**PROPOSED ALGORITHMS**	**16 - 17**
5.1	Introduction	16
5.2	Problem Definition and Input–Output Specification	16
5.3	Algorithms	16
5.3.1	Query Processing and Cultural Context Retrieval	16
5.3.2	Cultural Context Filtering and Ranking	16
5.3.3	Structured Prompt Generation	17
5.3.4	Image Generation and Cultural Evaluation	17
5.4	Evaluation Criteria	17
6	**PROPOSED FLOWCHART / DFD / BLOCK DIAGRAM**	**18 - 19**
6.1	Introduction	18
6.2	Overall System Block Diagram	18
6.3	Multilingual Query Processing and Cultural Knowledge Retrieval	18
6.4	API Request–Response and Exception Handling	19
6.5	Summary of the System Design	19
7	**IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)**	**20 - 23**
7.1	Introduction	20
7.2	Cultural Knowledge Retrieval and Context Filtering	20

7.3	Structured Prompt Generation	20 - 21
7.4	Models and Techniques for Image Generation	21
7.5	Main Pipeline Implementation	21
7.6	Backend and User Interface Implementation	22
7.7	Sample Input, Output and Implementation Limitations	22-23
7.8	Summary	23
8	**RESULT DISCUSSION AND ANALYSIS**	**24 - 27**
8.1	Introduction	24
8.2	Experimental Setup	24 - 25
8.3	Dataset and Test Query Description	25
8.4	Image Generation Results	25 - 26
8.5	Quantitative Metrics	26 - 27
9	**CONCLUSION AND FUTURE SCOPE**	**28 - 29**
9.1	Project Summary	28
9.2	Problem Addressed	28
9.3	Current Limitations	28 - 29
9.4	Potential Applications	29
9.5	Final Conclusion	29
10	**REFERENCES**	**30 - 31**

LIST OF FIGURES

Fig	Description	Page no.
1	Basic overall design	9
2	Proposed Methodology	11
3	Basic overall system block diagram	18
4	Multilingual Query Processing and Cultural knowledge retrieval	18
5	API Request Response flow diagram	19
6	SDXL base and refiner pipeline	21
7	During the Chhat puja celebration in Patna, a 30-year old woman offered sunset arghya at the river	25
8	During Durga Puja celebration in Kolkata, an elderly man in a white dhoti and kurta prays near a brightly lit pandal decorated with the marigold garlands and fairy lights.	26

LIST OF TABLES

Table No	Description	Page no.
1	Comparative Analysis of Existing Approaches and the Proposed Model	7
2	Software Configuration and Version Details for the Proposed System	24

ABSTRACT

Text-to-image (T2I) models built using latent diffusion and Diffusion Transformers can now generate highly realistic images. But they often fail to capture Indian cultural elements like attire, festivals, and rituals. These T2I models are mostly trained on English-centric and Western data, so they tend to generate stereotyped and culturally inaccurate images of Indian culture.

To address this problem, we propose a pipeline that combines Retrieval-Augmented Generation (RAG) and prompt engineering to generate culturally grounded images. The pipeline supports multiple Indian languages, such as Hindi and Punjabi, as well as English. Using RAG, we retrieve relevant information from Wikipedia and combine it with the user query. We then perform iterative multi-agent prompt refinement using a generation-feedback loop, in which the current prompt is continuously refined using feedback from the previous iteration.

We evaluate the proposed pipeline on the CUBE and C3 benchmarks across three criteria: cultural accuracy, cultural diversity, and prompt alignment. Cultural accuracy is measured using Cyclic Language-Grounded Correctness (CLGC), cultural diversity using the Weighted Vendi Score, and prompt alignment using Language-Grounded Correctness (LGC).

As future work, we plan to fine-tune our image generation model on the DRISHTIKON dataset to improve the T2I model's internal cultural knowledge. By doing so, we would also reduce the pipeline's dependence on RAG for retrieving cultural information.

1. INTRODUCTION

A text to image generation model is a machine learning model which takes an input natural language prompt and produces an image based on the input provided to the model.
It started emerging in the late 2010s and became more advanced rapidly in the early 2020s. In 2014 GANs were invented laying a foundation for image generation; later in 2016 GANs are successfully connected with text processing with models trained on small dataset which is generally domain specific, they started to generate small blurry images. Then in January 2021 a transformer system was announced which was DALL-E by OpenAI which acts as one of the major milestone in the development of text to image generation model; later DALL-E 2 was announced in April 2022 which was an upgraded version of the previous one generating more realistic images, followed by Midjourney in July 2022 and Stable Diffusion in August 2022 which started a race of becoming more advanced in the field on text to image generation by generating more and more realistic images.

As English being one of the widely used languages in the world, many text to image generation models often rely on English based prompts which basically acts as a limit for users who are not proficient in English. The basic solution for this problem is to use a translator as a medium of converting the regional or local language to English. However, the translators have some critical problems that can appear while dealing with regional languages such as Hindi, Haryanvi, Bengali, Tamil etc as they may fail to capture the regional terminologies and are lacking in cultural context and some ambiguous words that can have multiple meanings depending on the context.
India has a vast number of languages and traditions, and the cultural representation of the festivals are also based on the region they are celebrated. So, it becomes very important to accurately represent traditional clothing, rituals, ornaments and cultural symbols based on each religion.

Cultural groundedness in an image generated is basically the accurate representation of the cultural traditions, clothing, ornaments and rituals along with symbols and statues, the image should reflect the real world culture and regional context rather than focusing on visual appeal. Different regions and communities may have different traditions and different ways to follow the same culture so the model can combine different cultures which can happen due to less training data, prompt understanding and contextual knowledge. Therefore, cultural groundedness is important for improving the accuracy of cultural representation and reducing incorrect representation in images. This can be ensured using multiple evaluation criteria such as metrics.
Our goal is to generate a culturally grounded text to image generation model which generates images for Indian languages having regional features and context fully preserved, existing text to image generation models are mostly dependent on English as an input language limiting the accuracy of the image for the Indian language. Translating the language can also drop contexts and sometimes combine different cultures having the same terminologies due to less amount of cultural knowledge of ornaments, symbols, clothing etc.

So we need to create a context rich pipeline which is able to handle prompts from multiple languages and can retrieve related context from the data source and can pass the generated image from various evaluation metrics for the culturally right image generation.

India is a vast country having diverse languages, tradition and culture. People have different clothing habits, different ways to build their houses, different food habits likewise they have their own style to explain the things to AI, the user should be able to use their own language to define their culture and this is the gap we need to fill. The translation of the language sometimes cannot cover the same contextual meaning as it was in their own words. So improving cultural context in image generation is important to be covered and this project does the same by exploring a practical approach of making text to image prompting more culturally informed.

The main technical and fundamental objectives of this project are:

1. **Indian Language Acceptance:** The system accepts user queries in multiple Indian languages as input for culturally grounded text-to-image generation.
2. **Identifying Cultural Terms:** The system identifies and interprets culture-specific terms, ensuring their relevance to the intended cultural context of the generated image.
3. **Retrieving Context:** The system retrieves relevant cultural information from available knowledge sources to enrich the semantic understanding of the input query.
4. **Prompt Creation:** The system incorporates the retrieved cultural information into a structured, context-aware prompt to guide the image generation process toward culturally accurate and meaningful outputs.

The scope of this project is primarily limited to India allowing it to focus more on Indian cultural context, visuals, ornaments and symbols. The system accepts multi-lingual input for the model which is processed and combined with the culturally enriched, structured prompt to provide that as a final input to the pretrained model for image generation.The generated image will then be evaluated by selected metrics to ensure that the final image generated is actually culturally grounded or not. The entire process will be done with the help of a modular pipeline for easy error identification and future scalability.
The project focuses on improving cultural grounding through prompt engineering and contextual prompt generation rather than training a new text-to-image model from scratch.

2. LITERATURE REVIEW

2.1 Evolution of Text to Image Generation

Nowadays, the Text to image generation model has become more advanced making accurate and realistic images but in the earlier phase of image generation the image was generated by statistical modeling and pixel-by-pixel predictions and texture synthesis due to which they struggled to create realistic and high resolution images. Non-Parametric Texture Synthesis and Exemplar Based Method is one of the earlier approaches which helps in generating new textures and image regions by reusing and reordering the original patches from the existing images. Later learning based approaches were introduced by the Restricted Boltzmann Machines (RBMs) and Deep Belief Networks (DBNs) that captured patterns in data. Then Variational Autoencoders (VAEs) generate images by learning a continuous latent space and Autoregressive models by predicting image tokens sequentially. Then finally, Generative Adversarial Networks (GANs), introduced by Ian Goodfellow and colleagues in 2014.

2.2 Traditional Generative Models

Traditional generative models are the computer programs or the models which can learn patterns from the existing real images and try to create new images from scratch.
Few types of traditional image generation models are:

1. **Variational Autoencoders (VAEs):** Learns the data in a compact representation i.e a tiny summary of abstract numbers and tries to generate new images from the learned data.
2. **Generative Adversarial Networks (GANs):** Uses two networks one for image generation and another is used to check that the generated image looks accurate to the reality standard of the image.
3. **Autoregressive Models:** It predicts image tokens sequentially means it generates images step by step where each part depends on the previously generated part.
4. **RBMs and DBNs:** These generate images by learning the hidden patterns and relationship between the data.

There are some problems with these types of models such as slow and difficult training. Sometimes generates blur images and faces problems in creating complex images.
So, in order to overcome these situations Diffusion models are made.

2.3 Existing Text-to-Image Models

The existing text to image generation model works as a digital artist who learns through many large dataset in order to learn the relationship between text and the particular visual part of the image. These models are able to understand the natural language or the human language feeded to them as an input and tries to create the image based on the data they are trained on.

Some of the existing text to image generation models are:

1. **Stable Diffusion:** An open source latent diffusion model which basically performs the denoising of the data in the compressed, lower dimensional space instead of directly applying on pixel space and then tries to generate images based.
2. **DALL-E:** It is a text to image generation model powered by OpenAI which basically converts the detailed human language prompt to the respective image. It has evolved upto its 3rd version **DALL-E 3** which is the current version and it also has a lot of security filters to prevent nsfw content, violence and racism.
3. **Midjourney:** It is text to image generation model developed by **David Holz** founder of an independent research lab situated at San Francisco which is widely used for its creative & different visual styles and visually appealing images.
4. **FLUX:** It is an open-source model which is developed by **Black Forest labs**, it is based on advanced hybrid architecture **MultiModel diffusion transformers** and **Rectified Flow matching**. One of its power is that it has **Perfect Text Rendering** which is basically that it can write words and text inside the image without any spelling error.

Despite being powerful these models are not well riched with indian context and culture because of which it does not present the culture like clothings and ornaments accurate all the time. Due to lack of information and generic prompt these sometimes can mix culture and traditions.

2.4 Indian Language Processing and Multilingual Embeddings

India is a very vast country spreading over 3.28 million sq. km. following multiple language at different part of the country like Hindi, Haryanvi, Punjabi, Bengali and Tamil covering from north to south. Each language has its own grammar and vocabulary. Processing these regional language is very challenging as some words meaning depends on the context and cultural background of the region. Direct translation can sometime result in loss of cultural context of the words.

So in order to preserve the context of the Indian language we use Indic NLP. Indic NLP is used to do the computational linguistic work on the cultural language means it is used to understand and process the Indian language. We use IndicBERT type models to learn and understand patterns and language related information from the Indian language.

Multilingual Embeddings are used to convert text to numerical embeddings so that similar words become closer to each other and these embeddings are comparatively easy to understand for the model making image generation better. With the help of these multilingual embeddings we find the semantic similarity between the user input prompt and the available cultural description of the religion. However, the semantic similarity often does not mean that the cultural context is completely correct so that's why we need to retrieve culturally enriched data from known data sources and then with the help of user input and the data that is retrieved from the source we generate a structured prompt which acts as the final prompt for model input.

2.5 Retrieval-Augmented Generation (RAG)

Retrieval Augmented Generation is a type of technique which helps the user to generate better output responses by providing relevant information from external sources, Normally language models works only with the data they are trained on without any other source of data, So sometimes they just generate in-accurate data because of the less amount of data they are trained on whereas RAG fills that gap by providing the relevant data so that the responses generated can become better.

RAG pipeline basically consist of two main step:

1. Retrieving useful related information from the user prompt.
2. Generating output with the help of that retrieved information.

Keywords, semantic searches and text embeddings like techniques are used for context retrieval. Talking about cultural application, RAG can be used for gaining information about the festivals, ornaments, signs and symbols of different religions. The accuracy of the RAG depends on the quality of the available data and the relevance of the retrieved information.

Talking about our project, RAG is used to gain information regarding the culture of the Indian language prompt used and then creating a structured prompt which acts as a final input to the model.

2.6 Structured Prompt Generation

Structured prompt generation is a core part of prompt engineering. Normal prompts are just the basic description of the user requirement whereas the Structured prompt has a systematic way of organising the content of the image including subjects, clothing, ornaments, activities, background, lighting and visual styles.

Talking about Indian culture it is very important to clearly mention the regional traditions, traditional attire and clothing, surrounding environment etc. So that's why to mention these contents respectively we created structured prompts with the help of retrieved data from the data sources with the help of RAG and the normal prompt provided by the user.

Structured Prompt = Normal Prompt + Retrieved Information + Extra Details

However, structured prompts does not guarantee that the culture accuracy is achieved, the final generated image also depends on the capabilities of the model you are using because if the model is not trained on something relevant then the model can never configure about the representation of that particular wording because it can never interpret it.
So that means,

Structured Prompts ≠ Better result

However,

Structured Prompts + Relevant Cultural Context + Capable Model = Better Results

2.7 Existing Cultural Text-to-Image Datasets and Benchmarks

There are basically few Datasets/Benchmarks to check the culturally groundedness of the model, those are:

1. **CUBE (Cultural Benchmark for Text-to-Image Models):** This benchmark basically checks how well the text to image generation model represents different cultures accurately. Consisting of cultural information of about 8 countries including India as well covering cuisines, landmarks etc. It evaluates cultural awareness as well as cultural diversity.
2. **C3 (Challenging Cross-Cultural Benchmark):** Its main function is to check how well text to image generative models handles different cultural backgrounds related prompts. This benchmark helps in identifying the model limitation and evaluating the cultural representation.
3. **DRISHTIKON:** It specifically focuses on Indian culture, covering 15 languages over 64000 data rows consisting of festivals, landmarks, arts, cuisines etc. It focuses on multilingual and multimodal cultural understanding.

2.8 Comparative Analysis of Existing Approaches

Existing Approach	Author	Main Focus	Limitation
Existing multilingual text-to-image systems	Google, OpenAI (DALL·E), and Stability AI (Stable Diffusion)	Generate images from prompts in different languages.	Not specifically focused on preserving Indian cultural content.
CUBE Benchmark	Nithish Kannen et al. (2024)	Evaluates cultural competence of text-to-image models.	Evaluates cultural representation rather than directly enriching prompts for generation.
C3 Benchmark	Bingshuai Liu et al. (2023)	Evaluates cross-cultural text-to-image generation.	Does not specifically focus on Indian cultural context.
DRISHTIKON Benchmark	Arijit Maji et al. (2025)	Evaluates multilingual and multimodal understanding of Indian culture.	Focuses on evaluation rather than culturally enriched prompt generation.
Retrieval-Augmented Generation (RAG) systems	Lewis et al. (2020), for the foundational RAG framework	Retrieves relevant external knowledge to provide context to a language model.	Cultural accuracy depends on the relevance and quality of retrieved knowledge.
Proposed Model	Nishant and Anuj	Generates culturally enriched prompts using relevant information for Indian-language inputs.	Currently focuses on prompt engineering and contextual enrichment rather than training a new T2I model.

Table 1: Comparative Analysis of Existing Approaches and the Proposed Model

2.9 Research Gaps Identified

Existing models can generate images from multiple languages but no one is focusing on Indian culture itself, defining every minute details is challenging. Also, supporting some language does not guarantee that the model will correctly understand the cultural meaning of the prompt.

Benchmarks like CUBE and C3 generally check whether the generated image is culturally accurate or not, which does not help in generating the image itself.

RAG systems can retrieve relevant information from different data sources but it does not specifically focus on the Indian culture to guarantee the cultural resemblance with the real world. Therefore, the proposed project aims to combine multilingual input processing, cultural information retrieval, and structured prompt generation into a single pipeline to generate more culturally grounded and improved images.

Some of the questions that came to our minds while researching about them are:

1. What is the way in which Indian cultural concepts can be accurately depicted in images produced from Indian language prompts?
2. What is the way of preserving the cultural significance of Indian terms when converting multilingual input into a format that is understood by text-to-image models?
3. In images generated by AI, how is it possible to accurately represent the regional variations in Indian traditions, clothing, ornaments, and symbols?
4. What is to be done about the absence of Indian cultural context in existing text-to-image models without having to train a new model from scratch?
5. What ways are there of improving culturally accurate image generation when the existing models may produce images that are visually appropriate but fail to include important cultural details?

3. PROBLEM DEFINITION AND OBJECTIVES

3.1 Core Problem

The biggest gap I ran into when exploring these types of models is that we have many text to image generation models which can generate image based on the given input prompt and the data they are trained on, some of the models can accept multiple languages and that's why they are known as multilingual models. However, supporting the language does not guarantee that the image going to be generated will be culturally accurate.

India has a diverse range of traditions, clothing, symbols and ornaments and the minor details about these can be very challenging to represent accurately with the help of a mere prompt in there regional language. Direct translation of the regional language to the supported English or Western language is also a big problem because most of the time these translations result in loss of the cultural context present in the regional language.

In the same way, the cultural benchmarks just focus on the evaluation of the performance of the model by analysing the output image generated by the pretrained model on the limited amount of data about the religion. However, our main goal should be to increase the cultural information provided to the model before generating the image so that the image going to be generated will become highly polished regarding the cultural representation of the objects.

3.2 Formal Problem Statement

The main problem addressed in this project is the lack of culturally groundedness from the images generated by multilingual models for the Indian languages because of the less amount of culturally enriched data on which they are trained for that particular language. Translation also results in the loss of culturally grounded context. Therefore, there is a need to provide regional culture details to the model so that the quality of the generated image becomes enhanced.

My model system will mainly focus on processing any Indian language to retrieve relevant cultural information regarding those input and then creating a resourceful structured prompt which will be provided to the text to image generative model and helps them in generating the images more accurate to the culture.

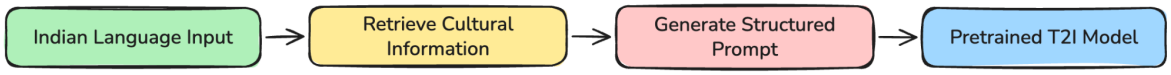


Figure 1: Basic overall design

3.3 Detailed Objectives

To actually solve this problem, I broke my project into six well-specified objectives:

1. **Processing the Multilingual input:** Building a module that is capable of processing the cultural description from the multilingual Indian languages given as an input.
2. **Retrieving the Cultural information:** Building a module that is capable of retrieving relevant information regarding the traditions, rituals, clothing, ornaments and many more from the cultural description provided in the input.
3. **Generating the contextual prompt:** Building a module capable of generating a structured prompt from the user input and the retrieved cultural information, resulting in creating a final prompt for the text to image generation model.
4. **Improving the prompt quality:** Building a module capable of denosing the generated contextual prompt by removing the unnecessary and contradictory details from the final prompt and including more relevant details into the final prompt.
5. **Pipeline Integration:** Building the entire pipeline capable of handling these previous modules in a systematic way.
6. **Evaluation:** Building a module for assessing the generated prompts thorugh suitable evaluation criteria and evaluating how accurately the result images represent the intended cultural context.

3.4 Project Scope and Boundaries

The main scope of our project is on the culture and regional traditions spread all over India with the help of prompt engineering and RAG rather than training a complete model from scratch, which includes processing the Indian language inputs, understanding the cultural description, retrieving cultural information regarding the culture, and then generating the structured prompt with the help of input and the retrieved data, which is given to the pretrained text to image generative model for the improved image generation.

Our main focus is on generating highly accurate images through the relevant structured prompt, i.e cultural prompt generation and prompt engineering rather than training a text to image model from scratch. Besides creating the culturally enriched prompt, the main generation of the image depends on the capabilities and understanding of the model, which is out of scope for this phase of project building.

4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 Introduction to the Proposed Methodology

Our proposed model is mainly focused on generating a structured prompt which will be culturally enriched to become the input for the text to image generation model. In the normal workflow generally the input prompt was directly provided to the image generation model, but if the prompt has a lack of information regarding the culture then the generated image can have wrongly depicted traditional clothes, ornaments, symbols.

So in order to address this problem gap we are proposing a modular pipeline solution in which the input provided by the user will be further processed to become a culturally enriched and enhanced prompt for the pretrained text to image generation model. This modular pipeline will consist of different stages including multiple Indian language input processing, cultural knowledge retrieval, context filtering and structured prompt generation. For the cultural knowledge retrieval we can use some external data sources like Wikipedia for instance.

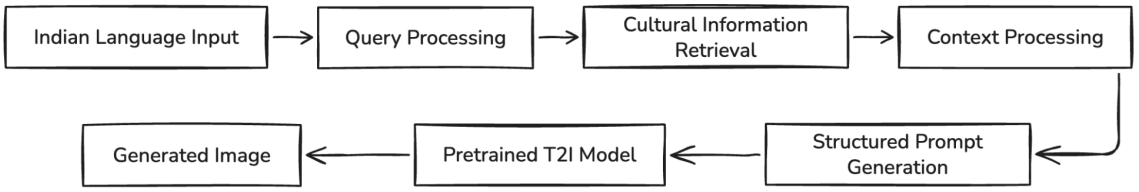


Figure 2: Proposed Methodology

4.2 Overall System Architecture

The main components of our proposed model are as follows:

1. **User Interface:** It will work as a platform for the user to input their respective query and get the desired generated image.
2. **Input Processing Module:** A module which will preprocess the query, remove noise from it and make it suitable for the later modules to understand.
3. **Language Processing Module:** It will try to understand the language, and also do the translation if required.
4. **Cultural Knowledge Module:** A module which will retrieve related information from the sources with the help of provided input.
5. **Context Processing Module:** A module to select the relevant details about the culture and the visuals from the retrieved information.
6. **Prompt Generation Module:** A module which will construct the structured prompt based on the retrieved knowledge and the provided input query.
7. **Backend and API:** It helps in making the overall pipeline work by creating the input-output endpoints and by connecting the different modules in the systematic way.

Example code for the proposed pipeline:

```
def generate_cultural_prompt(query):
    original_query = query

    processed_query = process_query(query)
    translated_query = translate_query(processed_query)

    context = retrieve_cultural_context(
        original_query,
        translated_query
    )

    relevant_context = process_context(context)

    prompt = construct_prompt(
        original_query,
        relevant_context
    )

    return prompt
```

4.3 Input Query Processing and Multilingual Translation

The first stage of the proposed system is to take the user's input query and get it ready for the next stage of processing. The user can input a description of the image they're looking for in any of the Indian languages that we support. Given that this is a project about Indian culture, the query may well contain regional words, names of festivals, traditional items and the like which we want to preserve.

Once the query has been received the system does some basic preprocessing work to clean up any extra spaces and formatting. The original query is also stored so that we don't lose any of the important cultural terms in the process.

Example Code for Normalisation:

```
def normalize_query(query):
    query = query.strip()
    query = " ".join(query.split())
    return query

user_query = "  दिवाली पर रंगोली  "
clean_query = normalize_query(user_query)

print(clean_query)
```

Next the system determines the input language and translates the query into English whenever necessary for future processing and retrieving information. There are some cultural terms (e.g. diya, rangoli, kalash) which if translated literally would lose their meaning so these can either be left in the original or mapped to an appropriate descriptive term.

```
cultural_terms = {
    "दीया": "traditional oil lamp (diya)",
    "रंगोली": "traditional rangoli floor art",
    "कलश": "traditional ritual pot (kalash)"
}
```

If translation fails the system can fall back on the original query and the available cultural term mappings. The query is then passed to the next stage where it is processed to produce a text representation and to retrieve the relevant cultural information. This allows the system to understand the query given by the user as input.

4.4 Text Representation and Cultural Knowledge Retrieval

Once a user sends the query, the system converts the text into a form to make meaning and gather the important words so that the system can understand the query. Then, the system searches for information related to the query, such as details from Wikipedia and other helpful sources. It could involve information related to festivals, traditional clothes, jewellery, religious practices, buildings and other cultural elements of India.

Then, the system checks whether the information is useful for the query. For instance, if user queries about Diwali, system may provide details like information about diyas, rangoli and home decorations. It gives more information in contrast to the short query.

When the required information is obtained, it is passed to the next step. Here, it uses the information to extract important information and prepare appropriate content for creating good prompts. This ensures the creation of meaningful and cultural-related prompts.

Basic knowledge-base lookup:

```
cultural_knowledge = {
    "diwali": [
        "traditional diyas",
        "rangoli decorations",
        "festive home lighting"
    ],
    "holi": [
        "coloured powders",
        "festival of colours",
        "festive outdoor gathering"
    ]
}
event = "diwali"
context = cultural_knowledge.get(event, [])
print(context)
```

4.5 Cultural Context Processing

Now, after getting the information regarding the culture of the input query, the proposed system will now select the relevant cultural details from the information which was more useful for the generation of the user's image. For example, if the user wants to generate the image of Indian's celebrating Diwali then the system will add the relevant details into the context such as sweets, rangoli, family together, crackers etc., these details mostly depend on the quality of the user input or the content retrieved from the data sources.

The main goal of the system is to add the context information related to the input query not just randomly adding any cultural information the system finds suitable, so the literal and cultural meaning on the query of the will be preserved and the context of the Indian culture will be accurately represented. Finally the retrieved and selected cultural details are passed to the next module.

Example Code for Duplicate removal:

```
def remove_duplicates(details):
    return list(dict.fromkeys(details))

details = [
    "traditional diyas",
    "rangoli decorations",
    "traditional diyas",
    "festive lighting"
]

filtered_details = remove_duplicates(details)
print(filtered_details)
```

4.6 Structured Prompt Generation

Once this module receives the cultural information generated by the previous modules then this will start working on generating a structured prompt with the help of the information provided to it i.e the original query provided by the user combined with the relevant information regarding that culture so the text to image model can clearly understand what to represent in the image.

The system can organise the prompt into different parts such as main subject, cultural elements, clothing, surroundings, colours and overall visual appearance which will make the prompt more clear and descriptive. And it will also make sure that the cultural words like diya, rangoli will not get replaced by providing the original user query.

Finally, the structured prompt will be generated by combining all these details to fulfill a sole purpose of providing better and accurate information to the text to image generation model. And then this prompt is passed down the module to the pretrained text to image generation model to get the highly culturally grounded image from the input provided by the user.

Code example of structured prompt:

```
def construct_prompt(subject, details, setting, style):
    cultural_details = ", ".join(details)

    prompt = (
        f"{subject}, "
        f"{cultural_details}, "
        f"setting: {setting}, "
        f"style: {style}"
    )

    return prompt

prompt = construct_prompt(
    subject="An Indian family celebrating Diwali",
    details=["traditional diyas", "rangoli decorations"],
    setting="decorated traditional home",
    style="detailed digital illustration"
)

print(prompt)
```

4.7 Experimental Methodology and Summary

The working of the system will be checked with various input queries in supported Indian languages. The generated prompt will be judged on parameters such as appropriateness and suitability of the prompt, ease to understand, Indian culture related information among others. Various examples will be used for this.

In general, this method aims to generate culturally relevant prompts through the generation of culturally relevant prompts from multilingual input and culture knowledge retrieval and prompt generation.

5. PROPOSED ALGORITHMS

5.1 Introduction

In this project we are building a modular pipeline which will work on the input Indian languages by processing them and understanding the culture, then it retrieve the cultural information from sources and then generates a structured prompt. All this process are divided into stages like language processing, cultural retrieval, context filtering and prompt construction.

5.2 Problem Definition and Input–Output Specification

- **Input:** Takes Indian language as an input text query.
- **Processing:** All the stages are processed in this part i.e, the language processing, cultural retrieval etc.
- **Output:** Culturally grounded image as an output

5.3 Algorithms

5.3.1 Query Processing and Cultural Context Retrieval

```
FUNCTION RetrieveContext(query)
INPUT: query = A text by user in an Indian language
OUTPUT: context = It is the cultural data relevant to the query
BEGIN
     1. Preprocess input query.
     2. Map Indic cultural data terms if available.
     3. Translate query or terms to be used for searching.
     4. Search within curated Indic cultural knowledge database.
     5. If needed, search Wikipedia for relevant data.
     6. Aggregate search results.
     7. Output available cultural context.
END
```

5.3.2 Cultural Context Filtering and Ranking

```
FUNCTION FilterContext(query, context)
INPUT: query = Processed user query
          context = Retrieved cultural information from previous
stage
OUTPUT: relevant_context = Selected cultural details
BEGIN
    1. Break the context into useful pieces.
    2. Select visual culture details.
    3. Select relevant culture categories, if applicable.
    4. Compute similarity or relevance scores, if applicable.
    5. Discard irrelevant or low-relevance pieces.
    6. Choose appropriate context pieces.
    7. Return relevant_context.
END
```

5.3.3 Structured Prompt Generation

```
FUNCTION GeneratePrompt(query, relevant_context)
INPUT: query = Original user query from the input
          relevant_context = Filtered cultural details from prev.
stage
OUTPUT: positive_prompt, negative_prompt
BEGIN
    1. Identify the main subject and activity.
    2. Combine the query with relevant cultural details.
    3. Organize details into structured prompt blocks.
         4. Add visual composition and style descriptors, if
applicable.
    5. Create a negative prompt according to rules.
    6. Return positive_prompt and negative_prompt.
END
```

5.3.4 Image Generation and Cultural Evaluation

```
FUNC: GenerateAndEvaluateImage(positive_prompt, negative_prompt)
INPUT: positive_prompt = Culturally enriched prompt
       negative_prompt = Prompt describing unwanted elements
OUTPUT: generated_image, evaluation_results
BEGIN
    1. Pass the positive and negative prompts to a
       pretrained text-to-image model.
    2. Generate an image using the provided prompts.
    3. Identify the expected cultural elements from the
       original query and retrieved cultural context.
    4. Analyze the output image with respect to:
       a. Level of presence of the culture
       b. Relation to the original query
       c. Presence of the expected elements
       d. Incorrect or mixed cultural elements
    5. Record the evaluation results using predefined
       criteria or a suitable evaluation method.
    6. RETURN generated_image, evaluation_results.
END
```

5.4 Evaluation Criteria

- **Cultural Accuracy:** We use this evaluation criteria to check how accurately the visuals of the image aligns with the real life.
- **Prompt Alignment:** We use this evaluation criteria to check how accurately the image aligns with the prompt provided.
- **Visual Element coverage:** We use this evaluation criteria to calculate what percentage of the expected items got represented in the output image.
- **Cultural Inconsistency:** We use this evaluation criteria to check whether the image includes unrelated or incorrectly mixed cultural elements.

6. PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

6.1 Introduction

There is a pipeline that this system follows. Starting with a query in Indian language as an input. Then, use that query to search for relevant information about Indian culture then filter relevant information and add the original query to it to produce a prompt. Use this prompt with a pretrained text to image model to generate images.

6.2 Overall System Block Diagram

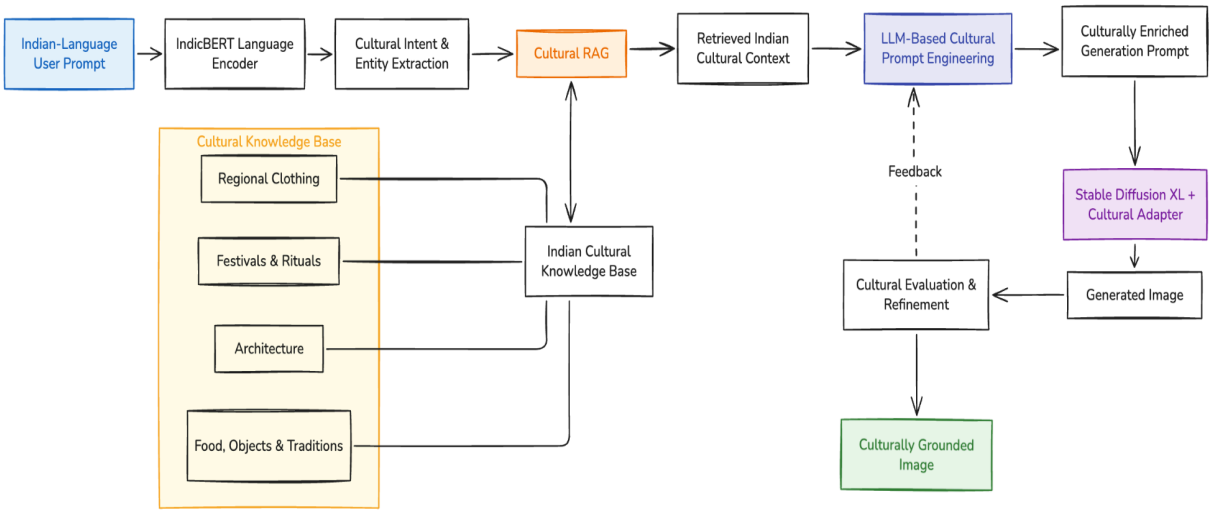


Figure 3: Basic overall system block diagram

6.3 Multilingual Query Processing and Cultural Knowledge Retrieval

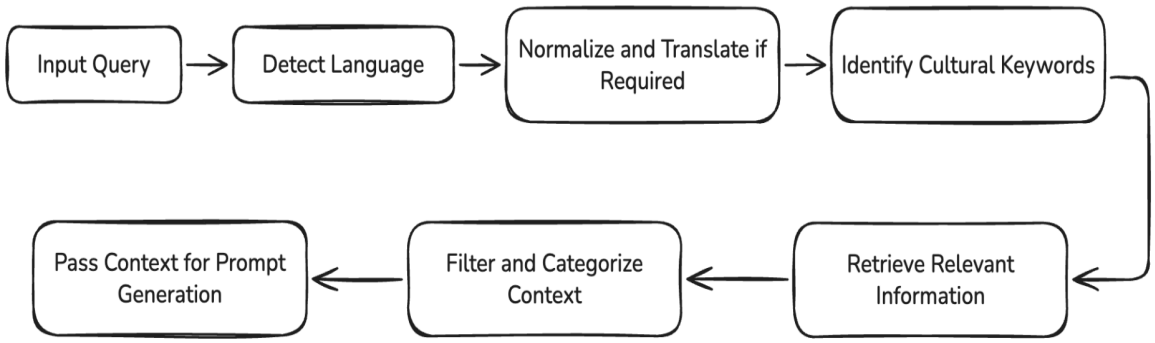


Figure 4: Multilingual Query Processing and Cultural knowledge retrieval

6.4 API Request–Response and Exception Handling

The API provides the user a way to request with the help of a query and get the response as a culturally enriched generated image.

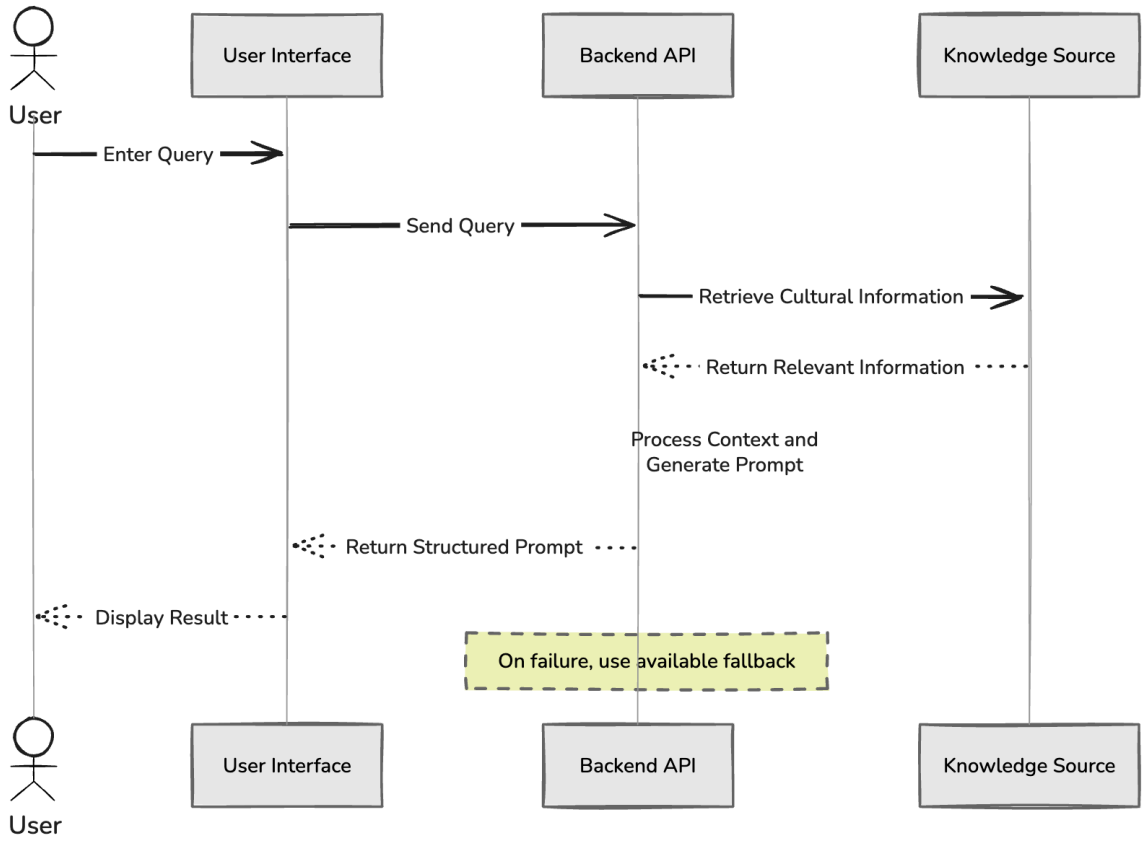


Figure 5: API Request Response flow diagram

6.5 Summary of the System Design

So overall, the system uses a pipeline to handle multilingual queries. The system pulls Indian cultural information. The system then creates prompts. The block diagrams and DFDs show how information moves between the parts. The flowchart and sequence diagram show how the system processes data and talks with APIs. This design gives the system a base for building and expanding the culturally grounded text-, to-image prompt generation system.

7. IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

7.1 Introduction

In this section we describe the tool and technology that we used to build Sanskriti T2I. Most Text-to-image models are predominantly trained on English and Western Data, As a result they often fail to capture the Indian cultural element like attire, festival and rituals and may produce generic or culturally inaccurate image of Indian culture. We introduce a new image generation pipeline Sanskriti T2I that try to solve this problem of cultural inaccuracy in T2I Models. Sanskriti T2I is a Retrieval Augmented, multi-agent prompt enrichment image generation pipeline design to improve the cultural fidelity of T2I Model. Our main objective is to accept the user input in Indian language and generate a culturally grounded image without relying on model internal knowledge about the culture. To achieve this we retrieve relevant cultural information using Wikipedia and Web Search and use it to produce culturally grounded prompt. Our Sanskriti T2I Pipeline has three major component: RAG, multi-agent prompt enrichment and image generation. In RAG we retrieve the relevant information of cultural keywords in user query using Wikipedia and Web Search. In multi-agent prompt enrichment we use AI agent to generate culturally grounded prompt using retrieved information and user query and then other AI agent will review the prompt and produce the feedback, this loop continue until max number of iteration are achieved. Then this refined prompt is passed to stable diffusion based T2I Model.

7.2 Cultural Knowledge Retrieval and Context Filtering

In this section we are going to discuss how we retrieve information from the web in our Sanskriti T2I model. Most T2I models are trained on Western data, as a result they fail to understand Indian cultural elements, so when they are asked to generate an image related to Indian culture they often generate generic or sometimes culturally inaccurate images. The main problem here is that the internal knowledge of T2I models about Indian culture is limited, and the knowledge they have is also very generic. To solve this problem we introduce a RAG architecture. RAG architecture was introduced to reduce hallucination of LLMs on topics that they are not trained on or have limited knowledge of. RAG solves this problem by providing sufficient knowledge about the topic to the LLM. In our pipeline we implement RAG so that it understands Indian cultural elements better and portrays them well. We use Wikipedia and web search using Tavily API to fetch relevant information about the keywords in the user query. We first extract keywords from the user query and convert them into a searchable query using a multilingual open-source LLM, the gemma3:4b model, to enhance the searching

7.3 Structured Prompt Generation

In this section we are going to discuss how we refine the prompt in our Sanskriti T2I pipeline after retrieving relevant information. After retrieving the relevant information, we combine it with the user query using the open-source model gemma3:4b to get an augmented query. After that, we generate an initial image using this query. Then we use the qwen3-vl:2b vision model to generate a description of the generated image. We pass this description, along with the user query, to qwen2.5:3b, an open-source LLM, and ask it to judge the image description on the basis of the user query, that is, whether it captures the cultural elements in the user query. The LLM provides a feedback score and feedback as structured output. Then we pass

this feedback to qwen2.5:3b to generate a new prompt on the basis of the previous prompt and the generated feedback. We continue this process until we reach the maximum number of iterations or the feedback score is greater than the final threshold.

7.4 Models and Techniques for Image Generation

In this section we are going to discuss the models and techniques we use to generate the images in our Sanskriti T2I. For Sanskriti T2I we use Stable Diffusion XL (SDXL), which is an open-source latent diffusion model by Stability AI. SDXL generates an image by starting from random noise in a compressed latent space. Then the Stable Diffusion model performs multiple denoising steps on the random noise to generate the final image. The denoising steps in the Stable Diffusion model are usually guided by the prompt that we provide to the model. SDXL is a two-stage pipeline consisting of two models, a base model and a refiner model. The base model first generates the image using the provided prompt and creates the objects and overall layout of the scene. Then the refiner model takes the output image of the base model and improves the fine details such as texture, facial features, and ornaments. Currently, we are running our SDXL model on a Google Colab server to generate images faster.

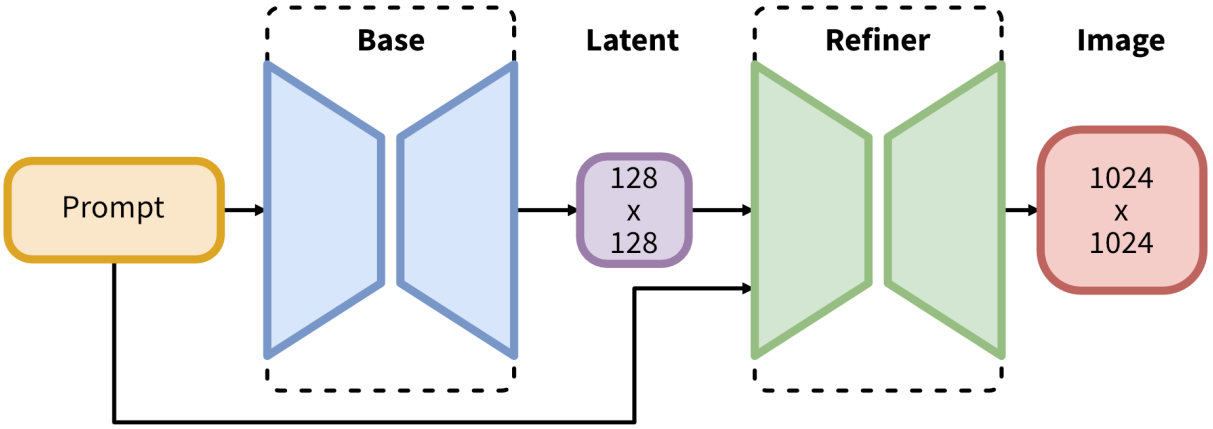


Figure 6: SDXL base and refiner pipeline .

7.5 Main Pipeline Implementation

Till now we have discussed all the components of the Sanskriti T2I workflow. Now we are going to integrate all the components into one pipeline. To do this, we are going to use LangGraph. LangGraph is an open-source Python framework used to build multi-agent applications. LangGraph is developed by LangChain Inc. to orchestrate workflows. Other than that, LangGraph also manages state persistence, checkpoints, and human-in-the-loop interaction. LangGraph orchestrates the workflow by building a graph and then executing the graph nodes. The graph nodes in LangGraph are user-defined Python functions, and the edges in LangGraph represent the control flow from one graph node to another. All the nodes in the graph share a common state in LangGraph.

7.6 Backend and User Interface Implementation

1.) For image generation, we use a Uvicorn server on Google Colab using FastAPI. This server exposes API endpoints like /generate-image, which takes a prompt from the client side and generates the image using the SDXL model.

We have implemented this because of resource constraints on our local machines. We cannot run the SDXL image generation pipeline because of the limited VRAM in our laptops, so we

run it on Colab servers with a 15 GB VRAM T4 GPU, which also provides us with fast inference.

2.) REST API Endpoints

- /generate-image: generates an image from the given user input. Expects a text field prompt.
- /generate-prompt: performs multi-agent iteration on the user input. Expects a text field prompt and a list of text fields for prompt.
- /run-pipeline: runs the entire pipeline, from retrieval to image generation. Expects the user query.

3.) We have also implemented Pydantic-based request and response schemas, so that we can validate the request fields for input validation and error handling

7.7 Sample Input, Output and Implementation Limitations

Sample input = "जयपुर के बाज़ार में एक चाय वाला, हाथ में केतली, पीछे रंग-बिरंगी दुकानें,",

Extracted Keyword = ['Jaipur', 'market', 'tea stall', 'kettle']

Extracted Context From wikipedia for Jaipur = Jaipur is the capital and the largest city of the north-western Indian state of Rajasthan. As of 2011,the city had a population of 3.1 million, making it the tenth most populous city in the country. Located 268km from the national capital New Delhi, Jaipur is also known as the Pink City due to the dominant colour scheme of its buildings in the old city.

Extracted Context from Tavily for Jaipur = As a fashion designer, Jaipur's block-printed fabric by the meter was a lot more enticing to me than its ready-made clothing, but I couldn't help but walk away with some of that also. Jaipur's famous textile is used for pajamas, button-downs, short-sleeve shirts, robes, dresses, saris, lehengas, an\nThe Nod recommends: Kassa for vegan bags and brass accessories, Anantaya for earthy table decor, Andraab for hand-spun pashminas, Tokree for breezy, kimono jackets and saris, Hot Pink for those who like a little pop of colour in their block-printed designs, Umrao Jewels for exquisitely handcrafted

Augmented prompt = A single [tea seller] stands in the Jaipur [market], holding a silver [kettle] with a wooden handle, his dark brown [pajamas] visible beneath. Behind him, a collection of shops displays vibrant, patterned [block-printed fabric] in shades of red, yellow, and blue, made of cotton. The [market] is illuminated by warm, golden light, reflecting off the polished brass [accessories] of a nearby stall.

Current Limitation in our workflows are:-

Noisy retrieval:- In the above example, Wikipedia retrieves the correct context, but Tavily extracts fashion blogs, which are not useful for this image generation task. In addition, there is noise in the above context, which can cause the model to hallucinate. To solve this, we can use a correction component that filters the noise in the context.

Generic content:- When we use Wikipedia and web search, the search returns mostly generic content. For example, in the case above, it returns geographic results instead of a description of how Jaipur looks. To solve this problem, we can increase the number of search results, or we can implement a component that will detect this issue, provide feedback on the search query, and then ask the model to rewrite the query using that feedback.

7.8 Summary

The Sanskriti T2I workflow can be divided into three components: RAG, multi-agent prompt refinement, and image generation.

In the RAG component, we use Wikipedia search and web search using the Tavily API to retrieve information about the keywords in the user query. The main limitation of this component is noisy retrieval, which can cause the model to hallucinate. One way to solve this problem is to use a corrective component that filters the noise from the context retrieved from Wikipedia. We use the gemma3:4b model to implement the RAG pipeline.

In the multi-agent pipeline, we first combine the retrieved context and the user query to produce an augmented query, which we then refine using a generate-feedback loop. The main limitation of this component is hallucination, where the LLM might add unwanted information that is not useful. One way to reduce this hallucination is to use a better prompt, a better model, or structured output. We use qwen2.5:3b and gemma3:4b to implement this multi-agent pipeline.

Finally, for image generation, we use the SDXL model running on a Google Colab T4 GPU with 15 GB VRAM to generate the image. We use the Hugging Face Diffusers library to download this model and to run inference on the Colab server.

8. RESULT DISCUSSION AND ANALYSIS

8.1 Introduction

Now that we have discussed the multiple components of our workflow, we will discuss how we evaluated it. Evaluation of generative models has become an active research area because of their open-endedness. When we try to evaluate the output of a generative model, we cannot directly compare it with a ground-truth reference; instead, we evaluate the output across multiple dimensions. In our project, Culturally Grounded Text-to-Image Generation, our main aim is to check whether the generated image is culturally accurate across multiple Indic languages. To do this, we have used three metrics, each of which evaluates a different aspect of the generated image. These metrics are Cyclic Language-Grounded Correctness (CLGC), Language-Grounded Correctness (LGC), and Weighted Vendi Score. Weighted Vendi evaluate the output diversity.

8.2 Experimental Setup

In this section we are going to describe the computational environment , software stack and model configuration.

Hardware Configuration:- We run all of our experiments on Google Colab T4 GPU with 15 GB VRAM and 12 GB system ram. We also use google drive to cache the model weight so we do not have to download the model weight every time we start a new session.

Software Configuration:-

Component	Version
Python	3.13.6
Pytorch	2.6.0
CUDA	13.4.3
diffusers	0.40.0
transformer	5.17.0
accelerate	1.15.0

Table 2: Software Configuration and Version Details for the Proposed System

Workflow configurations:-

Model Configuration: We are currently using the SDXL text-to-image model by Stability AI without fine-tuning, with 25 inference steps, a guidance scale of 7.5, and a random seed of 42.

RAG: We retrieve context using Wikipedia and web search, taking the top 3 results per query, and combine the user query with the retrieved context using Gemma3:4b.

Evaluation: We use Qwen2.5:3b, an open-source LLM, as a judge to compute CLGC and LGC, and we use the `vendi_score` library to compute the weighted version of the Vendi Score.

8.3 Dataset and Test Query Description

We use DRISHTIKON as our dataset to fine tune our T2I model. DRISHTIKON is a multimodal, multilingual benchmark for testing language models' understanding of Indian culture. It contains over 64,000 rows of text-image pairs. Unlike existing global benchmarks, DRISHTIKON focuses exclusively on India's cultural richness, spanning 15 languages, all 28 states, and 8 union territories. It covers 15 languages, with 14 Indic languages like Hindi, Punjabi, Bengali, etc., and English, and captures multiple Indian cultural elements like cuisine, dance and music, festivals, etc. We cannot use DRISHTIKON directly because it does not contain image captions; it contains multiple-choice questions which are related to images. So, we use Qwen3-VL:2B, an open-source vision model by Alibaba Labs, and ask it to generate an image caption of the image in DRISHTIKON. We ask it to generate image captions in English and then use Gemma3:4B to translate them into other languages.

To evaluate our pipeline, we use standard benchmarks: CUBE and C3 benchmark. CUBE is a dataset which was developed by Google Research to evaluate cultural awareness, and C3 was developed to evaluate cultural bias and evaluate how well generative models understand culture.

8.4 Image Generation Results

Here are some images that we generate using our workflow with the prompt we used.

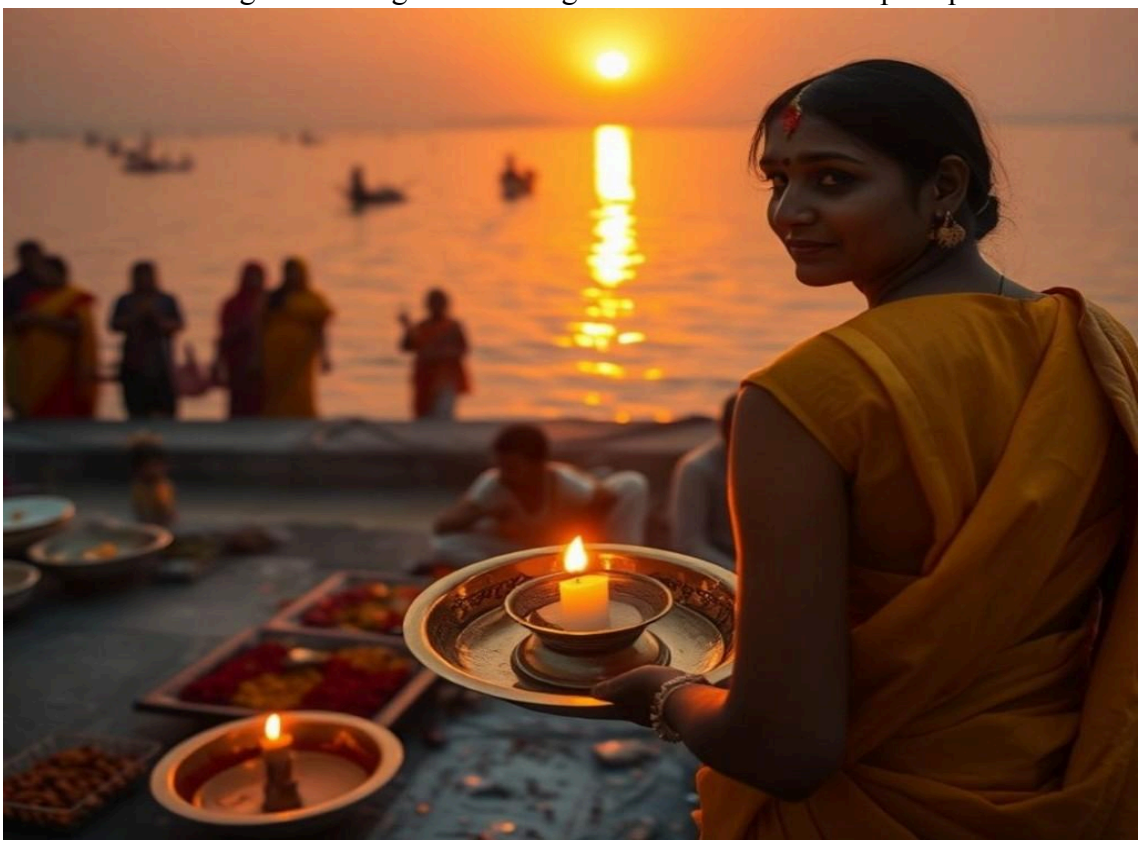


Figure 7: During Chhath Puja celebrations in Patna, a 30-year-old woman offered sunset arghya at the river.



Figure 8: During Durga Puja celebrations in Kolkata, an elderly man in a white dhoti and kurta offered prayers near a brightly lit pandal decorated with marigold garlands and fairy lights.

8.5 Quantitative Metrics

1.) **Cyclic Language-Grounded Correctness(CLGC)**: CLGC evaluates the generated images in text-text space. For each prompt in the benchmark, we generate n images. For each of the n images, a caption is generated using Qwen3-VL:2b. We then use Qwen2.5:3b as an LLM judge to evaluate each generated caption against the original prompt, scoring it from 0 to 10. Finally, the scores are averaged across all n images and all prompts to obtain the CLGC score for each language.

$$CLGC(l) = \frac{1}{n|P|} \sum_{p_k}^{P} \sum_{r=0}^{n} \phi[\, t(p_k),\ t(p'_k)\,]$$

Where **P** is the set of prompts, |P| is the number of prompts, and $t(p_k)$ is the k-th prompt and ϕ is the function that scores how well the image matches the English prompt. t(p'_k) denotes the generated caption for r th image.

2.) **Language Grounded Correctness(LGC)**: This metric evaluates the generated images in image-text space. For each prompt in the benchmark, we generate n images. Each generated image is then compared directly with the original prompt, using Qwen3-VL:2b as a judge, which scores how well the image matches the prompt from 0 to 10. Finally, the scores are averaged across all n images and all prompts to obtain the score for each language.

$$LGC(l) = \frac{1}{n|P|} \sum_{p_k}^{P} \sum_{r=0}^{n} \phi[\, t(p_k),\ f(i_{p_k})\,]$$

Where i_{p_k} denote the image generated by p_k prompt. $f(i_{p_k})$ denote function applied to the generated image i_{p_k}

3.) **Vendi Score**: This is a measure of image diversity. For each prompt in the benchmark we generate n images and compute the Vendi Score over that set. The Vendi Score takes the similarity matrix over the images and computes the effective number of distinct images in the set. The scores are then averaged across all prompts to give the score for each language.

9. CONCLUSION AND FUTURE SCOPE

9.1 Project Summary

In this project, we implement a workflow that produces culturally grounded images. We use technologies such as RAG, iterative prompt refinement, fine-tuning and evaluation to generate culturally grounded images. Along with culturally grounded image generation, our solution also works for many Indic languages, such as Hindi, Urdu, Assamese, Tamil and more. In our solution, we first retrieve information about the keywords in the user prompt through Wikipedia and internet search. We then pass all the retrieved information to a prompt refinement module, which refines the prompt iteratively using a multi-agent generate-feedback loop. We also evaluate our workflow across multiple Indic languages using metrics such as Cyclic Language-Grounded Correctness (CLGC), Language-Grounded Correctness (LGC), and Weighted Vendi Score.

9.2 Problem Addressed

Most image generation models are trained mostly on English and Western data. As a result, they often fail to capture Indic cultural elements like festivals, attire, rituals, or regional forms. The images generated by these models tend to be generic, stereotyped, or culturally inaccurate, and might not capture the visual representation of the culture. This matters because fields like education currently use AI-generated images for better visual understanding. If an image is inaccurate, learners might learn incorrect information. Governments use AI-generated images for awareness campaigns in regional languages, and culturally inaccurate images might reduce the trust and effectiveness of the campaign.

9.3 Current Limitations

Currently, our workflow can be culturally grounded using open-source image generation and an LLM (Large Language Model). But there are still some limitations in this workflow

- **Context retrieving from internet:** Our workflow relies on Wikipedia and Tavily Web search to extract the information of keywords in user query which in some cases where information regarding a keyword is very less on the internet. Our Wikipedia and Tavily retriever can retrieve wrong or useless information about the keyword which can directly affect the cultural accuracy of an image. One way to solve this issue is to integrate a corrective component which will check whether the retrieved information is correct and useful or not.
- **Hallucination:** Our workflow uses LLM(Large Language Model) for prompt refining. During Prompt refining the size of the prompt increases which can cause a small open source model with small context window to hallucinate. Open source models also have less support for Indic languages like Odia, Konkani which can also cause hallucination. One way to solve this problem is to use a bigger model with a large context window and good support for multiple Indic languages.
- **High Latency:** Retrieval augmented Generation(RAG), iterative prompt refining and image generation are the three major components of our workflow. Each of the

components have their latency bottleneck. In RAG bottleneck is searching for the information, in iterative prompt refining multiple iteration and multi-agent interaction is bottleneck and in image generation multiple denoising step is the bottleneck. All of these bottlenecks constitute High latency in our workflow.To fix this we can cache retrieval results or reduce the number denoising step.

9.4 Potential Applications

Our workflow can be applied in multiple area

- **Education:** Teacher, textbook publisher, and ed.tech platform can generate the culturally accurate image of attire, festival, and rituals for better visual understanding of the region. our implementation also work for multiple indic language so teacher from low-resource language communities can also use it using their mother tongue to generate images
- **Advertising and Public Communication:** Government and Ad agency can use it to generate the image for campaigns for a regional audience. if the image is not culturally accurate the regional audience might get offended which will reduce the effectiveness of the campaign.
- **Cultural Tourism:** Tourism boards and travel agencies can generate a culturally grounded image of attire, festival and ritual to promote the tourism of the destination and set realistic expectations for visitors.

9.5 Final Conclusion

In this project, we try to build a pipeline that takes a user query in an Indic language and produces a culturally grounded image. Most image generation models are trained on English and Western data, due to which they fail to capture Indic cultural elements like attire, festivals and historical monuments, and generate generic and culturally inaccurate images of them. There are image generation models trained on multiple cultural datasets, but there is still a problem with support for many Indic languages like Odia and Maithili. This is important in fields like education and advertising, where cultural accuracy matters. Our pipeline uses an existing state-of-the-art multilingual LLM (Large Language Model) along with web and Wikipedia search. We also implemented a prompt refinement technique, which refines the user prompt using the information retrieved from the web and Wikipedia, and then generates the final image using a Stable Diffusion model.

9.6 Future Scope

1.) **Fine-Tuning**: To improve cultural accuracy, we can fine-tune the model using LoRA (Low-Rank Adaptation). The main purpose of fine-tuning is to improve the model internal knowledge of Indian culture, so that our model does not rely completely on RAG for information.

2.) **Corrective Component**: We also plan to implement a corrective component to solve the problem of noisy retrieval. By doing so, we also reduce hallucination in the LLM we use later in the pipeline.

REFERENCES

[1] Surbhi *et al.*, "Navigating text-to-image generative bias across Indic languages," *IEEE Access*, vol. 12, pp. 45120–45132, Mar. 2024.

[2] Parth *et al.*, "When cultures meet: Multicultural text-to-image generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, 2024, pp. 11200–11210.

[3] Fulong *et al.*, "AltDiffusion: A multilingual text-to-image diffusion model," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5410–5422, Aug. 2024.

[4] Bingshuai *et al.*, "On the cultural gap in text-to-image generation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Singapore, 2023, pp. 3120–3131.

[5] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, 2021, pp. 8748–8763. *(Note: Formatted with standard author surname)*

[6] M. Mor *et al.*, "Navigating cultural chasms: Exploring and unlocking the cultural POV of text-to-image models," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 6, pp. 5820–5828, 2024.

[7] Suchae *et al.*, "Culture-TRIP: Culturally-aware text-to-image generation with iterative prompt refinement," *IEEE Robot. Autom. Lett.*, vol. 9, no. 5, pp. 4410–4417, May 2024.

[8] N. Kannen, "Beyond aesthetics: Cultural competence in text-to-image models," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Abu Dhabi, UAE, 2024, pp. 1540–1544.

[9] N. S. Mudiraj and S. Singh, "Semantic mapping of hindi text-to-image generation using CUB dataset," *Int. J. Comput. Appl.*, vol. 182, no. 12, pp. 22–27, Jan. 2023.

[10] Shravan *et al.*, "CulturalFrames: Assessing cultural expectation alignment in text-to-image models and evaluation metrics," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, 2025.

[11] Arijit *et al.*, "DRISHTIKON: A multimodal multilingual benchmark for testing language models' understanding on Indian culture," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Bangkok, Thailand, 2024, pp. 7890–7905.

[12] Nyasha *et al.*, "Culturally grounded text-to-image Gen-AI: LoRA fine-tuned stable diffusion models," *IEEE Access*, vol. 12, pp. 18200–18212, Feb. 2024.

[13] R. Rombach *et al.*, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, 2022, pp. 10684–10695.

[14] Mayur *et al.*, "Comparative study of pre-trained BERT and large language models for code-mixed named entity recognition," *ACM Trans. Asian Low-Resource Lang. Inf. Process.*, vol. 22, no. 4, pp. 1–18, Apr. 2023.

[15] C. Jeong, "Fine-tuning and utilization methods of domain-specific LLMs," *J. Inf. Process. Syst.*, vol. 20, no. 2, pp. 145–158, Apr. 2024.

[16] T. Brown *et al.*, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[17] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[18] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Comput. Linguistics (NAACL)*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[19] R. Jia *et al.*, "Mitigating cultural bias in text-to-image generative models via cross-lingual prompt alignment," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Milan, Italy, 2024, pp. 210–226.

[20] P. Dhariwal and A. Nichol, "Diffusion models beat GANs on image synthesis," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 8780–8794**.**

GitHub:

Respective link: https://github.com/nishant-0411/Sanskriti-T2I

The GitHub repository contains the project report, presentation slides, source code, and other relevant project materials. It also includes a README file with the necessary setup instructions, dependencies, and steps to run the proposed system, supporting reproducibility and evaluation.